

Student Performance Management System

Tools Used: Python · Dictionaries · Lists · Loops · Input
Validation

Prepared by: Sneha Mandlik

1. Problem Statement

Educational institutions often need a simple and efficient way to record student marks, calculate performance metrics, and generate marksheets. Manual calculation of totals and averages can be time-consuming and prone to errors.

This project aims to automate the process of student performance evaluation by:

- Recording marks for multiple students.
 - Calculating total and average scores automatically.
 - Generating structured marksheet reports.
-

2. Project Overview

Attribute	Details
Project Type	Python Console Application
Users	Teachers / Students
Data Stored	Student Names and Subject Marks
Calculations	Total Marks and Average Score
Output	Student Marksheet
Programming Language	Python

3. Methodology & Tools

The application was developed using Python fundamentals and data structures.

Tools & Concepts Used

- Python — Core programming language
- Dictionaries — Student record storage
- Lists — Subject-wise marks storage
- Loops — Multiple student and subject input
- Built-in Functions — `sum()`, `len()`
- Input Validation — Data accuracy checks
- F-Strings — Formatted output generation

Workflow

1. Accept number of students.
2. Record student names.

3. Record subject-wise marks.
 4. Store data in dictionaries.
 5. Calculate totals and averages.
 6. Display formatted marksheet.
-

4. System Design

Data Structure

Example:

Student Record:

Sneha → [85, 90, 88]

Krishna → [78, 82, 80]

Stored as:

```
{ "Sneha": [85,90,88], "Krishna": [78,82,80] }
```

Calculation Logic

Total Marks = Sum of Subject Marks

Average Marks = Total Marks ÷ Number of Subjects

5. Sample Output

Enter number of students: 3

Enter student name: Sneha

Enter marks for subject 1: 85

Enter marks for subject 2: 90

Enter marks for subject 3: 78

Enter student name: Amruta

Enter marks for subject 1: 72

Enter marks for subject 2: 88

Enter marks for subject 3: 91

--- Student Marksheet ---

Sneha => Marks: [85, 90, 78], Total: 253, Average: 84.33

Amruta => Marks: [72, 88, 91], Total: 251, Average: 83.67

6. Key Findings

1. Efficient Data Storage

Python dictionaries provide a structured method for storing student records and retrieving information quickly.

2. Scalable Design

The program can handle multiple students using loops and dynamic input collection.

3. Reduced Human Error

Automatic calculations minimize mistakes commonly found in manual marksheet preparation.

7. Future Enhancements

- Grade Calculation System
 - Pass/Fail Evaluation
 - CSV/Excel Export
 - GUI Development using Tkinter
-

8. Conclusion

The Student Performance Management System successfully automates the process of recording student marks and generating performance reports. The project demonstrates practical use of Python data structures, loops, built-in functions, and input handling techniques. It provides a

strong foundation for more advanced academic management systems involving databases, analytics, and visualization tools.
